

Consumo de APIs REST y GraphQL

Jhon Edison Prieto Artunduaga

Ingenieria de Software 2

24 de Abril de 2026

Índice

1. Parte 1: API REST (Rick and Morty)	2
1.1. Cuestionario REST	2
1.2. Proceso y Evidencias	2
2. Parte 2: API GraphQL (Countries API)	5
2.1. Cuestionario GraphQL	5
2.2. Proceso y Evidencias	5

1. Parte 1: API REST (Rick and Morty)

1.1. Cuestionario REST

- **¿Qué API elegiste y por qué?**
Elegí la API de Rick and Morty porque es muy amigable para empezar, no requiere procesos de autenticación y permite practicar búsquedas avanzadas mediante parámetros de consulta (*Query Params*).
- **¿Qué datos devuelve?**
Devuelve objetos en formato JSON con información detallada sobre personajes (especie, origen, estado), ubicaciones (tipo de planeta, residentes) y episodios.
- **¿Usa token o no? ¿Qué tipo?**
No utiliza ningún tipo de token, API Key o registro previo. Es una API pública de acceso totalmente libre.
- **¿Qué código de estado recibiste en cada request?**
En todas las peticiones recibí el código **200 OK**, confirmando que el servidor procesó la solicitud exitosamente.
- **¿Qué aprendiste diferente a JSONPlaceholder?**
Aprendí sobre la paginación real (la API incluye un objeto **info** con el total de páginas) y a combinar múltiples filtros en una misma URL usando el símbolo **&**.

1.2. Proceso y Evidencias

Se creó una colección en Postman para agrupar 5 peticiones GET hacia distintos endpoints. En cada una se programaron pruebas automáticas para verificar el código 200 y el formato de los datos recibidos.

1. GET Todos los personajes: Ejecución de la petición base para obtener la lista completa de personajes. Se comprobó el retorno de un arreglo de datos válido.

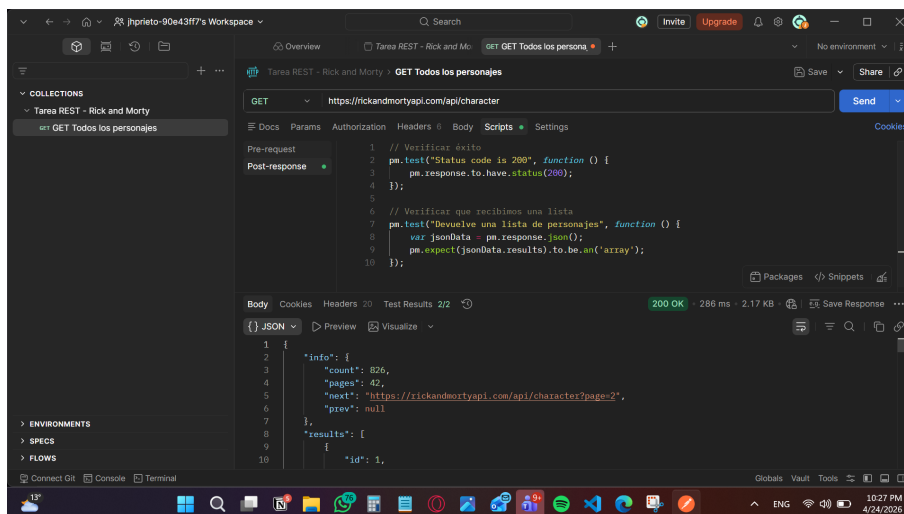


Figura 1: Petición para obtener todos los personajes.

2. GET Personaje por ID: Petición enfocada en obtener un único recurso filtrando por su ID específico directamente en la ruta de la URL.

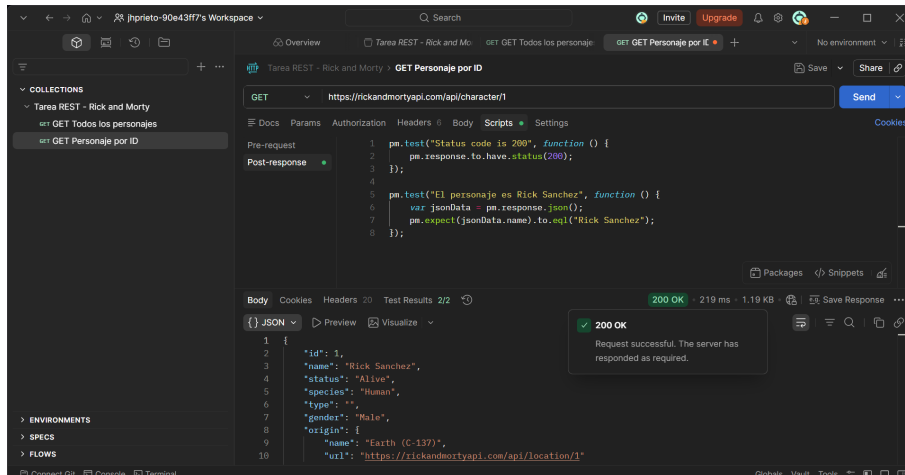


Figura 2: Consulta de un personaje mediante ID.

3. GET Filtrar por Morty Vivo: Uso de *Query Params* (`?name=morty&status=alive`) para realizar un filtro compuesto y obtener resultados específicos.

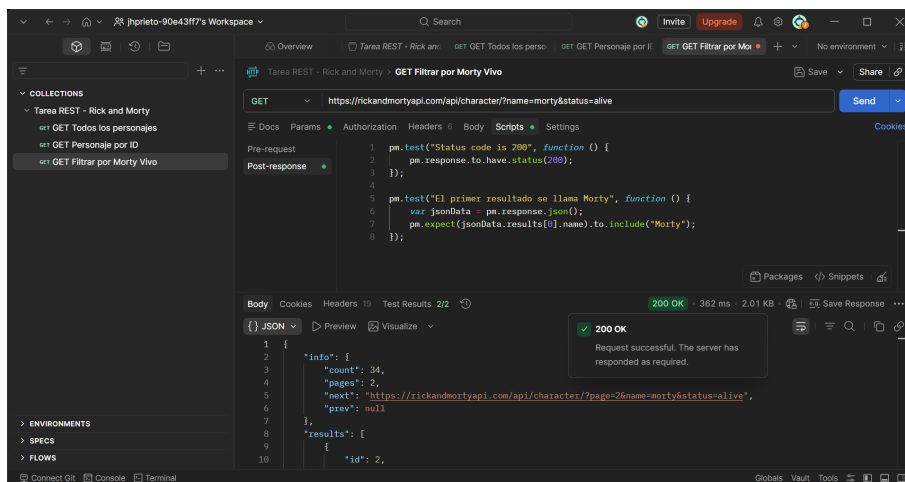


Figura 3: Uso de Query Params para filtrar personajes.

4. GET Ubicaciones: Petición al endpoint secundario de la API para obtener el listado de mundos y dimensiones.

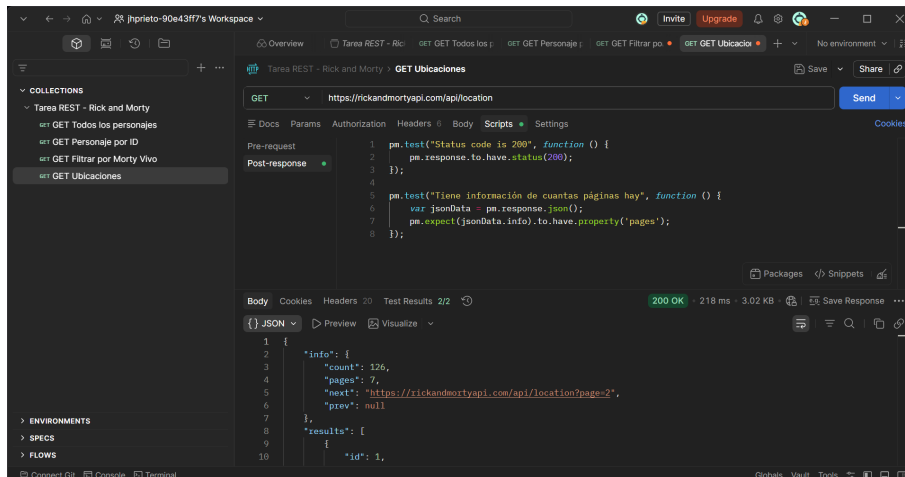


Figura 4: Listado de ubicaciones espaciales.

5. GET Episodios: Petición final al tercer endpoint principal de la API para consultar el catálogo de episodios de la serie.

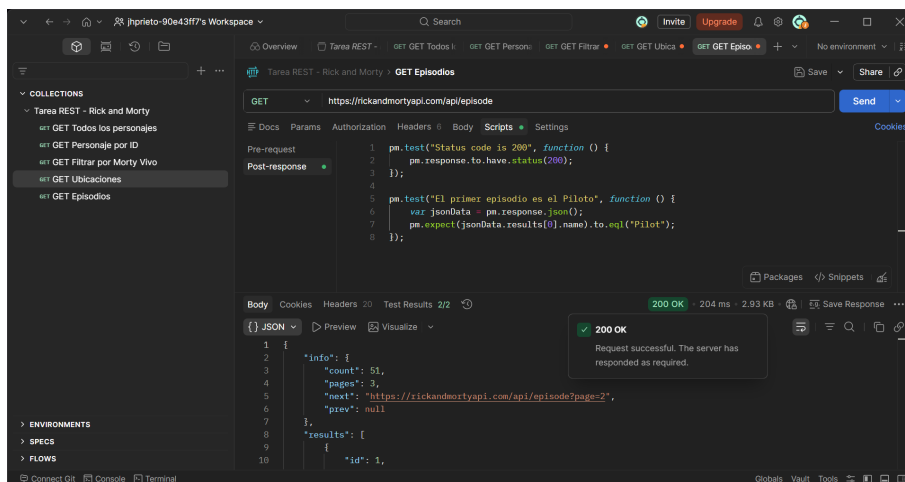


Figura 5: Consulta del catálogo de episodios.

2. Parte 2: API GraphQL (Countries API)

2.1. Cuestionario GraphQL

- ¿Qué diferencia encuentras vs REST?

La mayor diferencia es la eficiencia y el control. En REST se consultan múltiples URLs y suele haber *overfetching* (datos innecesarios). En GraphQL se usa una sola URL (método POST) y se pide una Query con la estructura exacta de los datos que se necesitan, recibiendo estrictamente eso.

- ¿Cuántos requests REST necesitarías para reemplazar tu query más compleja?

Para la query compleja (obtener un país, sus idiomas y sus estados a la vez), en REST habría necesitado al menos **3 requests separadas** apuntando a diferentes endpoints (ej. `/countries/CO`, `/countries/CO/languages`, y `/countries/CO/states`).

- ¿En qué proyecto real usarías GraphQL?

Lo usaría en aplicaciones móviles donde optimizar los datos consumidos es crucial, o en paneles de administración (*dashboards*) que requieran mostrar información cruzada de múltiples tablas en una sola carga.

2.2. Proceso y Evidencias

Todas las peticiones se configuraron con el método POST apuntando a un único endpoint. Dentro del cuerpo (*Body*), se estructuraron las *Queries* en formato GraphQL.

6. Query Simple - Continentes: Primera petición POST utilizando una *Query* básica para obtener la lista de continentes y sus códigos.

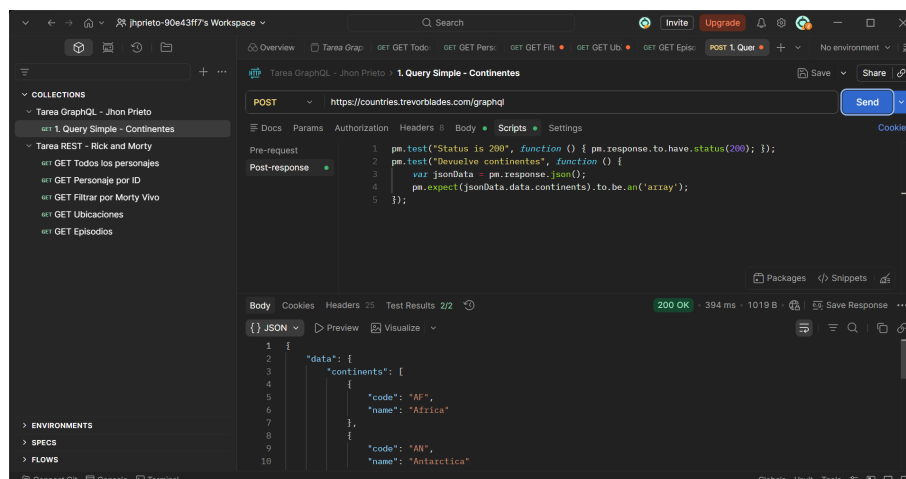


Figura 6: Listado simple de continentes.

7. Query Anidada - Países por Continente: Ejecución de una *Query* relacional, solicitando la lista de continentes y, anidado dentro de cada uno, los países que lo conforman.

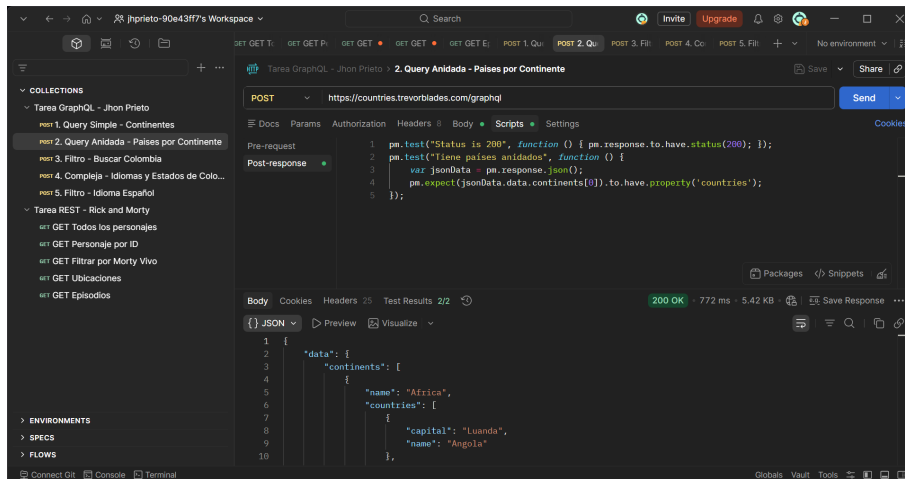


Figura 7: Datos anidados (Países dentro de continentes).

8. Filtro - Buscar Colombia: Uso de un filtro por argumento en la *Query* para extraer exclusivamente los datos del país con el código `CO`".

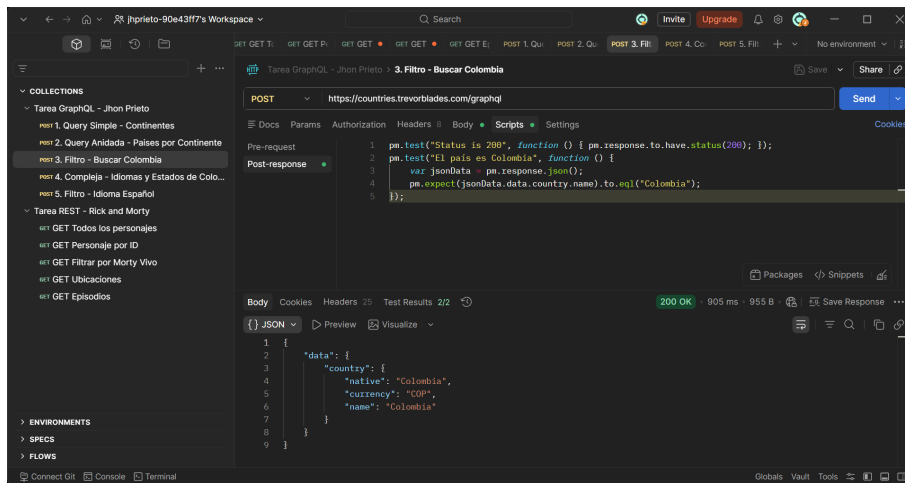


Figura 8: Filtrado de país por código.

9. Compleja - Idiomas y Estados: *Query* avanzada que combina el filtro por país con múltiples niveles de anidamiento, obteniendo lenguajes y divisiones políticas simultáneamente.

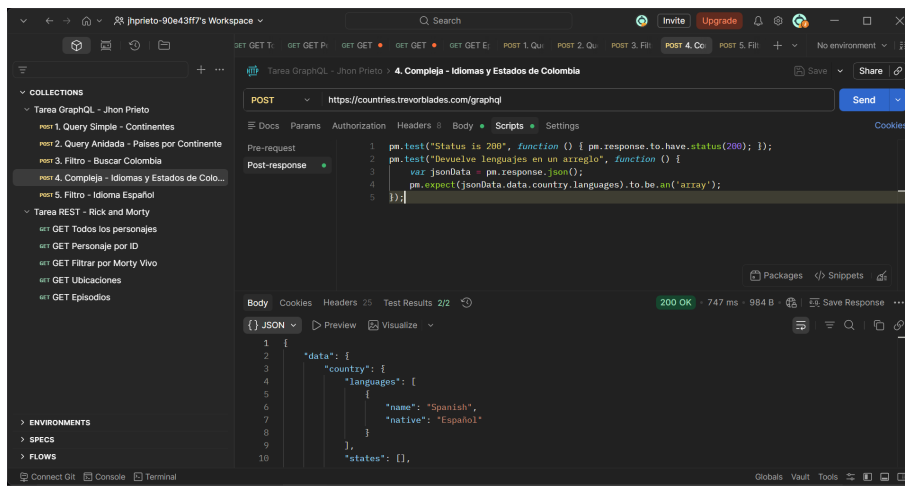


Figura 9: Query compleja con relaciones múltiples.

10. Filtro - Idioma Español: Petición final utilizando filtrado por el argumento de idioma (código `.es`) para recuperar sus metadatos nativos.

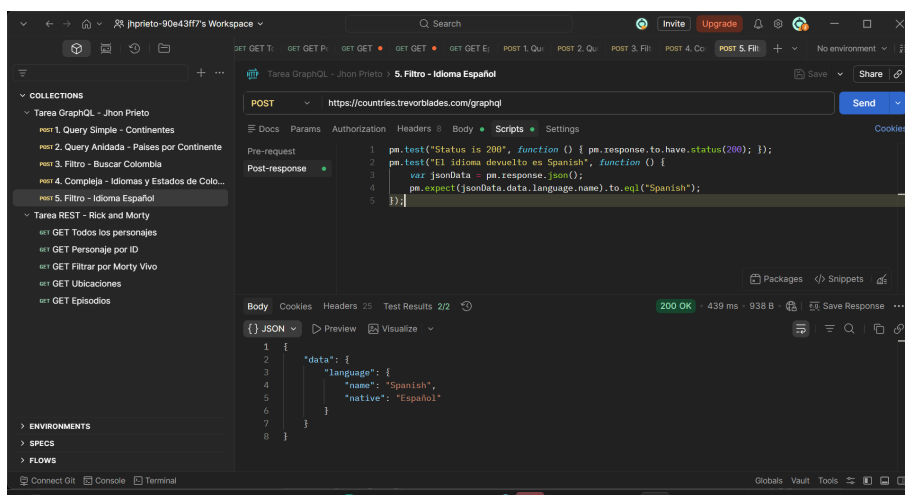


Figura 10: Búsqueda de metadatos mediante filtro de idioma.